

Day1 学习文档 v2.0 : AI Agent 到底是什么？

本文是《从零实现 Agent Runtime》学习阶段的 Day1 正式学习文档。

本文基于原始文件《AI Agent 学习笔记 (第一天) .pdf》重新整理，目标是把第一天的学习内容统一到 Day2 学习笔记的结构中，方便后续继续学习、复盘，以及未来改写成正式书稿。

与上一章的联系

Day1 是整个学习路线的起点，因此没有上一章。

这一章解决的是最基础、也最容易被误解的问题：

AI Agent 到底是什么？

在 Day1 之前，容易把 Agent 理解成“会调用工具的 ChatGPT”，或者把它想成某种中间件。但第一天的学习目标，是建立一个更工程化的认知：

Agent 不是单次模型调用，而是由 LLM、Runtime、Context、Tools/Skills 共同组成的执行系统。

Day1 的核心价值，是为后续学习 Runtime、Tool Calling、Memory、Human Approval、Streaming、MCP 和 Agent Framework 打底。

目录

什么是 Agent？

Agent 的核心循环

Tool 和 Skill 的区别

企业为什么要做自己的 Agent？

Agent 开发主要开发什么？

Agent Framework 是什么？

Agent Framework 最终产出的是什么？

AI 产品整体架构

AI 前端主要负责什么？

Conversation 和 Context 的区别

Memory

会话创建流程

Agent 后端负责什么？

前端真正难的地方

Streaming 为什么比普通 SSE 更复杂？

Agent 前后端职责划分

当前学习结论

写书 TODO

1. 什么是 Agent ?

最开始，Agent 可以被直觉地理解为一种“中间件”：

它承接 AI 需要做的事情，把人的工作流包装起来，交给 AI 去完成。

这个理解有一定道理，因为 Agent

确实经常位于用户、模型、工具和业务系统之间。但如果停留在“中间件”这个说法，会容易忽略 Agent 的核心：它不是单纯转发请求，而是在管理一个不断变化的执行过程。

更准确的理解是：

Agent = LLM + Runtime + Context + Tools / Skills

其中：

- LLM 负责推理、规划和决策。
- Runtime 负责驱动整个执行循环。
- Context 负责承载当前任务的上下文和状态。
- Tools / Skills 是 Agent 可以调用的外部能力。

这说明 Agent 不是一个单独的模型，也不是一个简单函数。它更像一个系统：模型只是其中负责推理的组件，真正让它持续工作的是 Runtime、Context 和 Tools 之间的协作。

2. Agent 的核心循环

Agent 本质上不是一次调用 LLM。

普通 Chat 往往是：

```
graph TD
    User --> LLM
    LLM --> Answer
```

而 Agent 是一个持续循环：

```
graph TD
    Goal --> LLM_Reason[LLM 推理 Reason]
    LLM_Reason --> Decision[决定是否调用 Tool]
    Decision --> Execute[执行 Tool Act]
    Execute --> Observe[获得结果 Observe]
    Observe --> UpdateContext[更新 Context]
    UpdateContext --> LLM_Reason
```

继续推理
|
v
直到完成

可以用一段极简伪代码表示：

```
while (!finished) {  
  const response = llm(context)  
  
  if (response.needTool) {  
    const result = tool()  
    context.push(result)  
    continue  
  }  
  
  break  
}
```

真正重要的是：

LLM 根据 Observation 动态决定下一步，而不是程序员提前把所有流程写死。

这也是 Agent 和普通业务流程编排最大的不同。传统流程是程序员定义完整路径，Agent 则是在 Runtime 的控制下，根据模型每一步的推理结果动态推进。

3. Tool 和 Skill 的区别

Tool 和 Skill 都是 Agent 的能力，但它们的粒度不同。

Tool

Tool 的粒度通常比较小，一个 Tool 基本对应一个明确能力。

例如：

- readFile
- writeFile
- Browser
- Shell
- Git
- SQL

Tool 更像底层能力接口。它们可以被 Runtime 调用，也可以被更高层的 Skill 组合起来。

Skill

Skill 通常由多个 Tool 组成，更接近业务能力或任务能力。

例如“修复 React Bug”这个 Skill，可能包含：

```
readFile()  
|  
v  
grep()  
|  
v  
terminal()  
|  
v  
writeFile()
```

|
v
npm test()

一个 Tool 解决一个动作，一个 Skill 解决一个任务。

因此可以简单理解为：

Tool = 原子能力
Skill = 多个 Tool 组成的任务能力

4. 企业为什么要做自己的 Agent ？

企业做 Agent 的目标，不是为了“超过 Claude Code”或“重新训练一个更强的模型”。

企业真正需要的是：

让 Agent 成为最懂公司的 AI。

通用模型不知道企业内部的信息，例如：

- CRM
- ERP
- OA
- Jira
- GitLab
- Jenkins
- 内部 Wiki
- 内部组件库
- 公司规范

所以企业 Agent 的核心架构往往是：

LLM
|
v
Company Runtime
|
v
Company Tools
|
v
Company Knowledge
|
v
Company Workflow

也就是说，企业 Agent 的价值不在于替代模型公司，而在于把通用模型接入企业自己的系统、知识和工作流。

5. Agent 开发主要开发什么 ？

开发 Agent，真正开发的通常不是 LLM。

LLM 是基础能力，企业和开发者更多是在它之上开发：

- Tool

- Context
- Runtime
- Memory
- Event Stream
- Human Approval
- Workflow

其中，Tool 在很多企业 Agent 中占比非常高。

例如：

- 查询订单
- 查询 CRM
- 查询 Jira
- 操作 Git
- 操作 Browser
- 发邮件
- 调 Jenkins
- 查库存

很多企业 Agent，80% 的代码可能都在 Tool 和业务系统适配层。

除了 Tool，Context 也非常关键。

Context 可能包括：

- 当前目标
- 历史聊天
- Tool 返回结果
- Memory
- RAG
- Prompt

LLM 每次收到的并不是“数据库里的完整世界”，而是 Runtime 构建出来的 Context。

Runtime 则负责：

- 调用 LLM
- Tool 调度
- Retry
- Timeout
- Human Approval
- Streaming
- Error Handle
- Memory

因此，Agent 开发的核心不是“训练大模型”，而是围绕模型搭建可执行、可控、可扩展的工程系统。

6. Agent Framework 是什么？

Agent Framework 的作用，可以简单理解为：

帮开发者写 Runtime。

常见 Agent Framework 包括：

- OpenAI Agents SDK
- LangGraph
- Mastra
- CrewAI
- AutoGen
- LlamaIndex

这些框架封装了大量 Runtime 相关能力，例如：

- Agent Loop
- Tool Calling
- Context 管理
- Memory
- Streaming
- Workflow
- 多 Agent 协作
- 错误处理

对于 TypeScript / Node.js 开发者来说，Mastra 和 OpenAI Agents SDK 会更容易上手，因为它们更贴近前端和 Node 全栈工程师的技术栈。

但 Day1 的学习结论不是“马上使用框架”，而是：

先理解 Runtime，再学习框架。

否则容易只会调用 API，却不知道框架为什么这样设计。

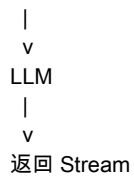
7. Agent Framework 最终产出的是什么？

Agent Framework 最终产出的不是一个 ChatGPT 网页。

更准确地说，它通常产出的是一个 Agent 服务。

典型结构是：

```
React
|
v
POST /chat
|
v
Agent Runtime
```



真正的聊天页面、消息列表、输入框、文件上传、Tool 状态展示，都需要前端自己开发。

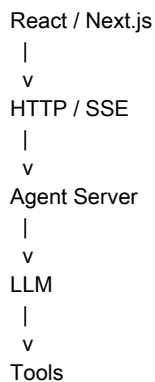
这意味着 Agent Framework 主要解决后端 Runtime 问题，而不是自动生成完整 AI 产品。

因此，做 AI 产品时需要同时理解两层：

- Agent Server 如何运行；
- React / Next.js 前端如何展示 Agent 的运行过程。

8. AI 产品整体架构

一个常见 AI 产品可以拆成三层：

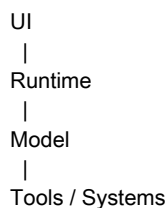


其中：

- React / Next.js 负责 UI。
- HTTP / SSE 负责请求和流式通信。
- Agent Server 负责 Runtime。
- LLM 负责推理。
- Tools 负责连接外部能力和企业系统。

这个结构说明，AI 产品不是只有“前端页面 + 一个模型接口”。

它更像一个完整系统：



对于前端工程师来说，这个架构很重要，因为它意味着 AI 前端不只是做聊天框，而是要理解后端 Agent 的事件和状态。

9. AI 前端主要负责什么？

AI 前端主要负责展示 Agent 的工作过程。

它包括：

- Chat UI
- 会话管理
- 文件上传
- Streaming
- Markdown 渲染
- Tool 状态
- Human Approval
- 多 Agent 展示
- Workflow 展示

传统前端展示的是“接口返回结果”。

AI 前端展示的是“Agent 正在如何工作”。

例如，一个 Agent 执行任务时可能经历：

```
Thinking
|
v
Planning
|
v
Tool Running
|
v
Need Approval
|
v
Continue
|
v
Done
```

这些状态需要前端用 Store、事件流、组件状态和 UI 反馈持续维护。

因此，AI 前端的复杂度不只是输入框和消息列表，而是要把 Agent 的执行过程清晰、可信、可控地展示出来。

10. Conversation 和 Context 的区别

Day1 已经初步区分了 Conversation 和 Context。

Conversation

Conversation 是数据库里的持久化数据。

例如：

```
conversationId
|
v
数据库
|
v
历史 Message
```

Conversation 用于保存真实会话历史。它通常包含用户消息、助手消息、工具消息等。

Context

Context 是每次请求时重新构建出来的模型输入。

它可能由以下部分组成：

```
System Prompt
+
Conversation Memory
+
User Memory
+
Knowledge (RAG)
+
当前消息
```

然后这些内容一起发送给 LLM。

LLM 不知道 conversationId，它只知道 Runtime 传给它的 Context。

因此：

```
Conversation = 数据库中的历史
Context      = 本次发送给 LLM 的输入
```

这个区分会在 Day2 进一步深化：Conversation 是事实，Context 是视图。

11. Memory

Memory 一般可以分为三层。

Conversation Memory

Conversation Memory 属于当前会话。

例如：

- 最近二十条聊天；
- 当前任务目标；
- 当前 Tool 执行结果；
- 当前对话内已经确认的信息。

User Memory

User Memory 跨会话存在，用于记录用户长期偏好。

例如：

- 用户偏好 TypeScript；
- 用户常用 React；
- 用户英文水平；
- 用户习惯的回答风格。

Knowledge

Knowledge 更偏向企业知识或外部知识。

例如：

- Wiki
- API 文档
- 设计规范
- 组件库
- 内部流程

这三层 Memory 服务于不同目标：

Conversation Memory = 当前任务连续性
 User Memory = 用户长期个性化
 Knowledge = 企业或领域知识

后续学习 Memory 时，需要特别注意：Memory 不等于数据库。Memory 的核心问题是 Runtime 如何在调用 LLM 前，把合适的信息放进 Context。

12. 会话创建流程

一个 AI 产品通常会有 Conversation 概念。

常见流程是：

```

  用户点击 New Chat
  |
  v
  React: POST /conversation
  |
  v
  后端生成 conversationId
  |
  v
  React 进入 /chat/{conversationId}
  
```

之后所有消息都会带上 conversationId。

另一种方案是延迟创建：

```

  第一次发送消息时 conversationId 为空
  |
  v
  Agent 后端自动创建 conversation
  |
  v
  返回 conversationId
  
```

两种方案都可以，取决于产品设计。

关键点是：

- Conversation 是产品层和后端存储层的概念；
- LLM 并不知道 conversationId；
- Runtime 会根据 conversationId 找到历史，再构建 Context。

13. Agent 后端负责什么？

Agent 后端主要负责 AI 运行相关的核心能力。

包括：

- Runtime
- Tool
- Memory
- Context
- LLM
- Event Stream

它不负责 React 页面本身。

这说明 Agent 后端和传统业务后端有相似之处，也有不同之处。

相似之处：

- 都需要接口；
- 都需要鉴权；
- 都需要存储；
- 都需要错误处理。

不同之处：

- Agent 后端需要管理 LLM 调用；
- 需要处理 Tool 调度；
- 需要构建 Context；
- 需要输出事件流；
- 需要支持 Human Approval；
- 需要维护 Agent 的执行状态。

因此，Agent 后端更像一个 AI Orchestrator，而不是普通 CRUD 服务。

14. 前端真正难的地方

AI 前端真正难的地方，不是文件上传。

真正难的是状态管理。

因为 Agent 不是一次返回结果，而是一个持续执行的过程。

例如：

```
Thinking
|
v
Planning
|
v
Tool Running
|
v
Need Approval
|
v
Continue
|
v
```

Done

前端需要维护这些状态，并将它们映射成用户能理解的 UI：

- 当前是否正在思考；
- 当前是否在调用 Tool；
- Tool 调用了哪个；
- Tool 执行成功还是失败；
- 是否需要用户审批；
- 最终回答是否完成；
- Stream 是否还在继续。

这使得 AI 前端更像一个实时状态机 UI，而不仅是传统的表单和列表页面。

15. Streaming 为什么比普通 SSE 更复杂？

普通 SSE 往往只有简单的消息流：

```
message
|
v
message
|
v
message
```

但 Agent 返回的是事件流。

例如：

```
thinking
|
v
tool_start
|
v
tool_finish
|
v
message_delta
|
v
approval_required
|
v
finish
```

这意味着前端不能只把所有内容拼成一段文本。

前端需要根据不同事件类型更新不同 UI：

- thinking 更新思考状态；
- tool_start 展示工具开始执行；
- tool_finish 展示工具执行结果；
- message_delta 流式追加模型回答；
- approval_required 展示审批控件；

- finish 标记任务完成。

因此，Agent Streaming 比普通聊天流更复杂。

它不是文本流，而是事件流。

16. Agent 前后端职责划分

Day1 最后明确了 Agent 前后端的职责边界。

Agent 后端负责

- Tool 执行
- Runtime
- Context
- LLM
- Event Stream
- Memory
- 权限判断
- Human Approval 流程控制

React 前端负责

- 接收 Event
- 更新 Store
- Streaming 展示
- Tool UI
- Approval UI
- Chat UI
- Workflow UI

前端并不知道 Tool 内部如何执行。

它只负责展示 Tool 的状态和 Agent 的执行过程。

职责划分可以总结为：

Agent 后端负责执行与调度。
React 前端负责展示与交互。

这个边界非常重要，因为它决定了后续开发 mini-agent-runtime 时，哪些能力属于 Runtime，哪些能力属于 UI。

17. 当前学习结论

Day1 的学习最终形成了以下核心结论。

第一，Agent 不只是 Tool Calling，而是 Runtime 驱动下的推理循环。

第二，企业开发 Agent 的重点不是训练模型，而是连接企业内部系统和知识。

第三，Agent Framework，例如 OpenAI Agents SDK、Mastra、LangGraph，主要帮助开发者实现 Runtime。

第四，Conversation 是数据库里的数据；Context 是每次请求动态构建后发送给 LLM 的上下文。

第五，AI 前端的核心不只是聊天框，而是展示 Agent 的执行过程、管理复杂状态和处理事件流。

第六，Tool 的执行发生在后端；前端负责接收事件并展示 Tool 的执行状态。

第七，对于有前端和 Node.js 经验的工程师来说，学习 Agent 更多是在学习 Runtime、Context、Tool，而不是训练大模型。

Day1 的一句话总结是：

Agent 不是一次模型调用，而是由 Runtime 驱动的“推理 - 行动 - 观察”循环。

18. 写书 TODO

未来把 Day1 改写成正式 Chapter 时，需要补充以下问题。这里只列问题，不写答案。

- Agent 一词的历史来源是什么？
- Agent 与 Chatbot 的区别需要用更多案例说明。
- Agent 与 Workflow 的区别是什么？
- Tool Calling 为什么不等于 Agent？
- ReAct 模式需要补充更完整的背景和示例。
- Runtime 在不同框架里分别叫什么？
- OpenAI Agents SDK、Mastra、LangGraph、CrewAI、AutoGen 的定位需要做对比表。
- 企业 Agent 的典型应用场景需要增加案例，例如客服、研发、运维、销售、财务。
- Tool 与 Skill 的边界需要给出更多工程示例。
- Memory 三层结构需要补充更正式的定义。
- Conversation 与 Context 需要增加一张对照表。
- Agent 产品前后端架构需要画成更正式的系统架构图。
- Streaming Event 需要补充事件协议示例。
- Human Approval 需要补充为什么属于 Runtime 控制。
- AI 前端状态管理需要补充 Store 设计示例。
- 为什么前端工程师适合学习 Agent Runtime，需要单独展开。
- 为什么学习路径应该先写 mini-agent-runtime，再学 Mastra / OpenAI Agents SDK，需要补充论证。
- Day1 与 Day2 的衔接需要更自然：从“Agent 是什么”过渡到“Runtime 才是大脑”。
- 后续写书时需要加入 Interview 问答，例如“Agent 和 Tool Calling 有什么区别？”
- 后续写书时需要加入 Source Code Mapping，对照真实框架源码中的 Runtime、Tool、Memory 概念。

19. 写书素材

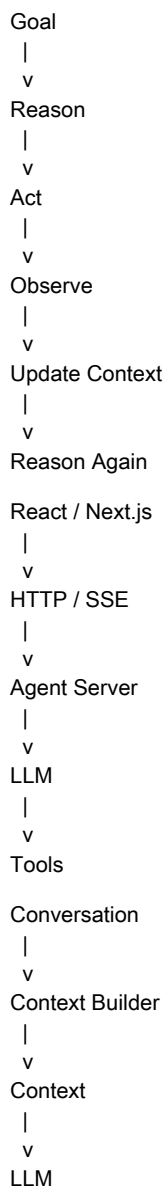
以下内容可以在未来写成正式 Chapter 时复用、扩展或改写。

核心句子

- Agent 不是一次模型调用，而是一个由 Runtime 驱动的执行系统。
- Agent = LLM + Runtime + Context + Tools / Skills。
- Tool 是原子能力，Skill 是任务能力。
- 企业做 Agent，不是为了重新训练模型，而是为了连接企业系统、知识和工作流。
- Agent Framework 的本质，是帮助开发者实现 Runtime。
- Conversation 是数据库里的持久化历史，Context 是每次请求前动态构建的模型输入。
- LLM 不知道 conversationId，它只知道 Runtime 给它的 Context。
- AI 前端不是只做聊天框，而是展示 Agent 的执行过程。
- Agent Streaming 不是普通文本流，而是事件流。
- Agent 后端负责执行与调度，React 前端负责展示与交互。

可复用架构图

Agent = LLM + Runtime + Context + Tools / Skills



可复用代码骨架

```
while (!finished) {  
  const response = llm(context)  
  
  if (response.needTool) {  
    const result = tool()  
    context.push(result)  
    continue  
  }  
  
  break  
}
```

可复用术语

- Agent
- Runtime
- Context
- Conversation
- Tool
- Skill
- Memory
- Event Stream
- Human Approval
- Agent Framework

可复用比喻

- Agent 不是一个聪明函数，而是一套会循环执行的系统。
- Tool 像螺丝刀，Skill 像“修好一个柜子”的完整能力。
- Conversation 像数据库里的历史记录。
- Context 像每次递给 LLM 的材料包。
- Agent Streaming 像后端持续广播 Agent 的执行事件。
- AI 前端像 Agent 执行过程的仪表盘。

未来章节衔接

Day1 建立了 Agent 的宏观认知。下一步自然进入 Day2：

- Agent 为什么不是一次 LLM 调用？
- Runtime 为什么是 Agent 的核心？
- LLM、Runtime、Tool 到底谁负责什么？
- Conversation 和 Context 的区别为什么会影响 Runtime 设计？

这些问题将引出 Day2：Agent Runtime —— 谁才是真正的大脑？